

Background: Smart Parking Barrier System.

A local university wants to implement a smart parking barrier system at its main gate. The system should:

- Automatically open the barrier for registered staff and students.
- Count the number of cars in the parking lot.
- Display the number of available spots.
- Trigger an alert if someone tries to enter when the parking is full.

The goal is to simulate and implement the logic using the Integrated Problem-Solving Process.

Step 1: Understand and Define the Problem (Analyse)

- The university needs an automated parking barrier system that controls vehicle entry and keeps track of parking availability. The system must identify whether a driver is a registered staff member or a student. If the user is registered and there is an available parking space, the barrier should open and allow the vehicle to enter. The number of cars inside the parking area should then increase.

If the parking lot has reached its maximum capacity, the barrier should remain closed and the system should display or trigger a parking full alert.

The system also needs to display the number of available parking spaces at all times.

For example, if the parking capacity is 100 cars and there are currently 65 cars, then:

$$\text{Available spots} = 100 - 65 = 35$$

Step 2: Organise and Describe the Data

Features: Automate access, vehicle count, alert for full capacity.

Input: RFID/ID of user, sensor for car entry/exit, max capacity output barrier control(open/close)

Output: Each vehicle has unique ID, sensor is reliable, capacity fixed.

Assumptions:

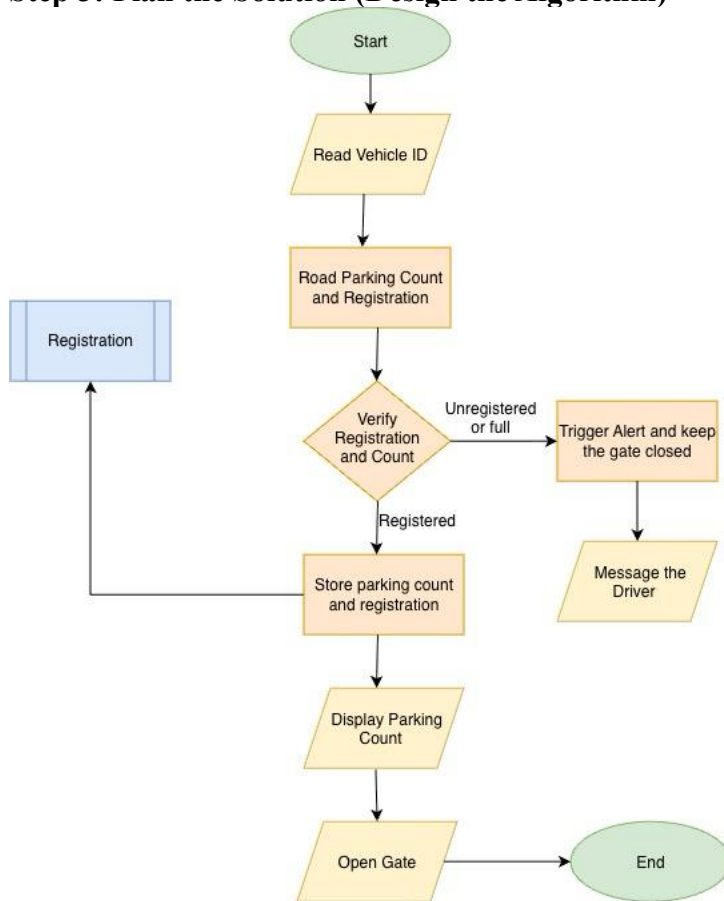
- Each registered user has a unique RFID/ID.
- Sensors and RFID reader work correctly.
- Parking capacity is fixed at 100.
- Registration data and vehicle count are accurate.

Expected outcome: Registered users can enter when spaces are available. If full or unregistered, entry is denied. The system displays available spaces.

Sketch: Block Diagram showing RFID reader – logic – Registration Database – Capacity Check - Barrier – Display + Alert

Parameter	Type	Example Value
Max Capacity	Constant	100 Vehicles
Current Count	Variable	75 Vehicles
Vehicle ID Input	Input	A12345
Is Registered?	Decision	Yes/No
Barrier Control	Output	Open/Close
Display Available	Output	25 spaces available
Alert Trigger	Output	ON/OFF

Step 3: Plan the Solution (Design the Algorithm)



Step 4: Implement the Solution (Word Coding)

- Once upon on time...
- Read vehicle ID (input)
- Check if the vehicle is registered.
- If not registered, slam the boom gate over the bumper of the car (that is, do not open the gate).
- If registered but carpark is full (always happens at university), do not open the gate and gently message the driver the number of spots.
- If registered and there are enough spots, open the gate, yey...

- Note that all transactions are being stored in a database called “Registrations”

Step 5: Test and Refine the Solution (Debug and Verify)

Manual Test Cases

Assumptions:

- Max number of cars allowed: 100
- Registrations database Is up to date (actual car park spots availability)

Test for....

- Registered user with 95 cars – Allows entry
- Registered user with 101 cars – Alert + Entry Locked + Message
- Unregistered user – Alert + Entry Locked + Message

Suggested improvement

- Add logging feature for entries
- Integrate web dashboard for monitoring
- Add override for emergency vehicles

Part 5: AI Agent Integration

In this part of the tutorial, you will explore how Artificial Intelligence (AI) can assist in solving problems, refining logic, and enhancing your documentation. You are encouraged to use an AI agent such as Microsoft Copilot to support your work.

Use Copilot to assist with at least two or more of the following:

1. Refine your logic or Word Code: Ask Copilot to review your Step 4 implementation and suggest improvements or identify potential issues.

```
BEGIN
  READ vehicle_ID
  LOOKUP vehicle_ID in Registrations database

  IF vehicle_ID NOT FOUND THEN
    SET Barrier = CLOSED
    DISPLAY "Access Denied: Unregistered Vehicle"
    TRIGGER Alert
  ELSE
    IF Current_Count >= Max_Capacity THEN
      SET Barrier = CLOSED
      DISPLAY "Parking Full - No spaces available"
      TRIGGER Alert
    ELSE
      SET Barrier = OPEN
      INCREMENT Current_Count
      DISPLAY (Max_Capacity - Current_Count) + " spaces available"
      LOG Entry in Registrations database
    ENDIF
  ENDIF
```

```
ENDIF  
END
```

Improvements Copilot Identified

- Removed humour (“slam the boom gate”) and replaced with realistic system behaviour.
- Added logging (matching your “Suggested improvement”).
- Ensured display updates after every transaction.
- Ensured alerts only trigger when needed.

2. Generate alternative solutions: Prompt Copilot to propose different ways to solve the problem or enhance your flowchart logic.

Alternative Solutions Proposed by Copilot

Copilot generated multiple ways to structure the system logic beyond the simple IF-ELSE approach.

A. Event-Driven Logic

- RFID Scan Event
- Capacity Check Event
- Barrier Control Event
- Display Update Event

This modular approach improves debugging and scalability.

B. State Machine Approach

States include:

- IDLE
- VERIFYING_USER
- CHECKING_CAPACITY
- OPENING_GATE
- DENYING_ENTRY

This is ideal for embedded systems like Arduino.

C. Queue-Based Logic

Vehicles enter a queue and are processed one at a time — useful when multiple cars arrive simultaneously.

3. Explore real-world implementation: Use Copilot to discuss how your system could be built using actual hardware (e.g., Arduino, Raspberry Pi).

Copilot explored how the system could be built using actual hardware.

Hardware Components

- RFID Reader (RC522 or PN532)
- Ultrasonic/IR sensors for entry/exit
- Servo motor for barrier arm
- Raspberry Pi or Arduino
- LCD/LED display
- Buzzer or LED alert system
- SQLite or cloud database

How It Works in Practice

1. RFID reader scans ID.
2. Raspberry Pi checks registration in database.
3. Sensors confirm vehicle presence.
4. Pi checks capacity.
5. Pi sends signal to servo → open/close barrier.
6. Display updates available spaces.
7. Entry is logged automatically.

Optional Enhancements

- Emergency override button
- Web dashboard (Flask/Node.js)
- Mobile app for staff/students
- Number plate recognition (ANPR camera)

4. Improve documentation: Ask Copilot to help you write a professional README.md file or summarize your project for presentation.

Copilot produced a professional README.md suitable for GitHub or assignment submission.

README.md — Smart Parking Barrier System

Overview

The Smart Parking Barrier System is an automated solution designed for a university environment. It verifies registered users, tracks vehicle count, displays available spaces, and triggers alerts when the car park reaches full capacity.

Features

- Automatic barrier opening for registered users
- Real-time vehicle counting
- Live display of available parking spaces
- Alerts for unregistered users or full capacity
- Logging of all entries
- Optional web dashboard

System Workflow

1. User scans RFID/ID
2. System checks registration
3. System checks parking capacity
4. If valid and space available → barrier opens
5. If invalid or full → barrier remains closed + alert
6. Vehicle count updates
7. Display shows remaining spaces

Hardware Components

- RFID Reader
- Ultrasonic/IR sensors
- Servo motor barrier
- Raspberry Pi / Arduino
- LCD/LED display
- Buzzer/LED alert system

Software Components

- Python or Arduino C++
- SQLite / Cloud database
- Optional Flask dashboard

Example Logic

Code

```
IF registered AND space_available:  
    open_gate()
```

```
ELSE:  
    deny_entry()  
    trigger_alert()
```

Future Improvements

- Emergency vehicle override
- Mobile app integration
- AI-based number plate recognition